Anthropic AI: Company, Claude Models, and AI Safety

Two-page educational document for testing a Retrieval-Augmented Generation (RAG) application

Page 1: Introduction to Anthropic AI

1. What Is Anthropic?

Anthropic is an artificial intelligence company founded in 2021. The company focuses on developing reliable, interpretable, and steerable AI systems. Its founders and early team included former OpenAI employees. Anthropic develops the Claude family of large language models (LLMs), which are designed to assist with writing, coding, research, data analysis, and problem-solving.

Anthropic aims to build AI systems that are useful, safe, and aligned with human intentions. It emphasizes AI safety research, model evaluation, and understanding how advanced AI systems behave.

2. What Is Claude?

Claude is a family of AI assistants and language models developed by Anthropic. Like other large language models, Claude processes text and generates responses based on patterns learned during training and the context provided in a conversation. Depending on the model and available tools, Claude can summarize long documents, explain technical concepts, generate software code, analyze information, and work with supported images or other input formats.

A language model does not automatically know every new event or every private document. Its answers depend on its training, the context it receives, and any external tools or information sources connected to it.

3. How Does Claude Work?

Claude uses transformer-based neural network technology. A transformer processes sequences of tokens and uses attention mechanisms to model relationships between elements in the input. The model generates output by predicting subsequent tokens based on the preceding context and its learned parameters.

The user submits a question or instruction.

The system converts the input into tokens.

The model processes the tokens using its neural network.

The model generates a response token by token.

The application returns the response to the user.

Real applications may add safety checks, retrieval-augmented generation (RAG), tool calls, and output validation. These steps depend on the application and are not necessarily performed in every interaction.

4. What Is Constitutional AI?

Constitutional AI is an approach developed by Anthropic to help train AI systems to follow a set of principles. These principles guide the evaluation and improvement of model responses. The approach can involve generating responses, critiquing them against stated principles, and using feedback to improve model behavior. It has also been associated with methods that reduce dependence on direct human feedback for every individual training example.

Constitutional AI does not guarantee that a model will always behave correctly. Models can still make mistakes, misunderstand instructions, or generate inaccurate information. Testing, monitoring, and additional safeguards remain important.

5. What Is AI Safety?

AI safety refers to research and engineering practices intended to reduce harmful, unreliable, or unintended behavior in AI systems. Activities include evaluating model performance and dangerous capabilities, testing responses to malicious or misleading instructions, studying interpretability, developing safeguards, and monitoring deployed systems.

AI safety matters because a model can produce convincing but incorrect answers. Applications should verify important information rather than treating every generated response as fact.

Page 2: Claude Applications, APIs, and RAG

6. Claude API and Developer Integration

Anthropic provides an API that developers can use to integrate Claude models into applications. Examples include customer-support assistants, coding assistants, and document-question-answering systems. A typical application has a frontend, a backend, and a model API integration. The frontend collects a request; the backend validates it, applies access controls, sends appropriate input to the model, and returns the processed response.

Developers should protect API keys using environment variables or a secret-management service. API keys should not be embedded in public frontend code or committed to public Git repositories. API access, supported model names, pricing, rate limits, and available features can change over time, so developers should consult Anthropic's official documentation.

7. What Is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) combines information retrieval with language-model generation. It allows an application to answer questions using external documents rather than relying exclusively on the model's learned knowledge.

For example, if a user uploads a document about Anthropic AI and asks, "What is Constitutional AI?", a RAG system searches the document collection for relevant passages, retrieves matching chunks, and supplies them as context to a language model. The model then generates an answer grounded in the retrieved information.

Document ingestion: Upload files or collect text from approved sources.

Text extraction: Convert supported files into searchable text.

Chunking: Divide the text into smaller, optionally overlapping sections.

Embedding generation: Convert each chunk into a numerical vector.

Vector storage: Store embeddings and associated text or metadata in a database.

Retrieval: Represent the user's question and find relevant chunks.

Generation: Send the question and retrieved context to the language model.

Response validation: Check relevance, access permissions, and citations where required.

RAG quality depends on document quality, chunk size, embedding quality, retrieval strategy, and the language model's ability to use retrieved context.

8. How Supabase Can Be Used in a RAG Application

Supabase provides a managed PostgreSQL database and supports the pgvector extension for storing and searching vector embeddings. It can serve as the storage layer for a RAG application.

A database table named documents might contain these fields:

id: A unique identifier for each document chunk.

content: The text of the chunk.

embedding: The numerical vector generated by an embedding model.

metadata: Additional information such as filename, page number, and document category.

During upload, the application extracts text, splits it into chunks, generates embeddings, and stores the chunks and vectors in Supabase. When a user asks a question, the application generates a query embedding and searches for similar vectors. The retrieved text is passed to the selected language model to generate an answer.

Important implementation details include matching the database vector dimension to the embedding model, protecting private documents with authorization and Row Level Security (RLS), and selecting an appropriate vector-search strategy. Supabase stores the data configured by the developer; it does not automatically understand or answer questions about every uploaded document. The application must implement ingestion, embedding generation, retrieval, and model integration.

9. Example RAG Questions for This Document

In which year was Anthropic founded?

What is Claude?

How does a transformer-based language model generate text?

What is Constitutional AI?

Does Constitutional AI guarantee completely correct answers?

What are the main stages of a RAG pipeline?

What is the purpose of embeddings in a RAG system?

How can Supabase and pgvector support semantic search?

Why should API keys not be placed in public frontend code?

What happens after a RAG system retrieves relevant document chunks?

10. Conclusion

Anthropic develops Claude models and conducts research focused on building useful and safer AI systems. Claude can be integrated into applications through supported developer interfaces, while RAG can provide external knowledge to improve document-grounded question answering. Combining a language model, an embedding model, and Supabase vector storage provides a practical foundation for a document-question-answering application. Secure database policies, reliable retrieval, appropriate model selection, and systematic testing are essential for a robust implementation.

Document note: This is a simplified educational overview prepared for RAG testing. Verify time-sensitive product details against Anthropic's official documentation.